

中心仓-前置仓即时零售网络的 供应链中断状态设计与马尔可夫仿真实验

学生任务说明文档

适用于：1 个中心仓、10 个前置仓、5 类产品、固定 (s, S) 策略的基线仿真模型

摘要

本文档用于指导学生在已经完成的中心仓-前置仓日度库存仿真模型之上，引入供应链中断风险、构造有限状态马尔可夫过程，并评估固定 (s, S) 库存策略在不同中断频率、严重程度和持续时间下的运营绩效。建议第一阶段只考虑两个外生风险因子：供应商可用性和供应商至中心仓的上游运输状态。每个因子设置四个状态，联合形成 $4 \times 4 = 16$ 个中断状态。文档给出状态含义、示例转移矩阵、参数解释、矩阵校验方法、仿真实验规则、伪代码、实验方案、评价指标和学生交付要求。文中数值是教学用示例，不代表经验估计；研究重点是建立可解释、可复现、可进行敏感性分析的建模流程。

关键原则

本阶段不要尝试把库存水平、缺货、利润和所有中断风险都塞入一个小型转移矩阵。正确做法是把库存系统状态与外生中断状态分开：库存由仿真模型内生演化，中断状态由有限状态马尔可夫链外生驱动。

目录

1	任务目标与最低完成标准	3
1.1	学生需要完成的核心任务	3
2	首先澄清：供应链“有多少种状态”？	3
2.1	库存系统状态与中断状态不是同一个概念	3
2.2	外生风险与内生结果的区分	4
3	本网络可能面对的中断风险	4
4	推荐的第一阶段马尔可夫状态模型	5
4.1	风险作用位置	5
4.2	供应商可用性链 A_t	5
4.3	上游运输状态链 U_t	6
4.4	中心仓库存位置必须包含所有未到货数量	6
4.5	联合中断状态：16 种组合	7

5	转移矩阵的设计、解释与校验	7
5.1	教学用示例矩阵	7
5.2	不要混淆频率、严重程度与持续时间	8
5.3	平稳分布与示例矩阵的长期含义	8
5.4	如何构造低、中、高风险情景	9
6	如何把中断过程接入现有仿真模型	10
6.1	每日事件顺序	10
6.2	推荐数据结构	10
6.3	完整伪代码	11
6.4	基础实现约定	11
7	实验设计	12
7.1	实验 A: 强制中断事件	12
7.2	实验 B: 长期马尔可夫运行	12
7.3	核心绩效指标	12
7.4	韧性指标	13
8	可选扩展与状态数量增长	13
8.1	下游运输中断	13
8.2	需求激增	14
8.3	状态爆炸问题	14
9	学生交付清单与建议工作流程	14
9.1	交付清单	14
9.2	建议的三阶段工作流程	15
10	常见错误检查表	15
A	用于矩阵校验的 Python 示例	16
B	用于状态抽样与管道更新的 Python 骨架	17

1 任务目标与最低完成标准

本项目的研究问题可以表述为：

在固定 (s, S) 库存策略不变的条件下，供应中断和运输中断的发生频率、严重程度、持续时间及其组合，将如何影响中心仓-前置仓即时零售网络的服务水平、库存成本、缺货损失、利润和恢复能力？

1.1 学生需要完成的核心任务

学生任务

- T1.** 列出与本网络有关的中断风险，并说明哪些风险适合第一阶段、哪些风险适合后续扩展。
- T2.** 为供应商可用性和上游运输分别定义有限状态集合及运营含义。
- T3.** 构造两个转移矩阵，检查每行概率和、平均持续时间和平稳分布。
- T4.** 将两个风险链与现有日度仿真模型连接起来，保持原 (s, S) 策略不变。
- T5.** 开展强制中断实验和长期马尔可夫实验，并用多次独立重复得到均值和置信区间。
- T6.** 解释哪些中断对哪些产品、哪些前置仓以及哪些绩效指标影响最大。

最低可交付版本只需要完成以下配置：

$$Z_t = (A_t, U_t), \quad |\mathcal{Z}| = 4 \times 4 = 16,$$

其中 A_t 表示供应商可用性， U_t 表示供应商到中心仓的运输状态。下游城市配送、中心仓作业中断和需求激增可以作为后续扩展。

2 首先澄清：供应链“有多少种状态”？

2.1 库存系统状态与中断状态不是同一个概念

现有仿真模型已经包含大量随时间变化的运营变量，例如中心仓库存、前置仓库存、在途订单、销售量和缺货量。把这些变量写成一个向量，可记为

$$X_t = (I_t^C, I_t^F, P_t^C, \text{orders}_t, \dots).$$

由于库存数量是整数且容量较大，完整运营状态 X_t 的可能组合数量非常庞大，没有必要逐一枚举。

本任务需要另行定义一个低维的外生中断状态：

$$Z_t \in \mathcal{Z}.$$

完整仿真状态是

$$S_t = (X_t, Z_t).$$

其中 X_t 由库存流、订单流和需求实现内生决定； Z_t 按马尔可夫过程演化，并改变供应释放量、运输提前期或作业能力。

常见误区

以下变量不应被直接设为外生马尔可夫状态：中心仓缺货、前置仓缺货、lost sales、低 fill rate、低利润、库存利用率过高。它们应当是供应中断、运输延误、需求和库存策略共同作用后由仿真产生的结果。

2.2 外生风险与内生结果的区分

表 1: 适合设为外生状态的风险与应由仿真产生的结果

类别	适合设为外生状态	不应设为外生状态
供应端	供应商产能下降、停产、良品率下降	中心仓最终缺货量
运输端	额外延误 1 天、额外延误 2 天、线路停运	某前置仓最终 fill rate
仓内作业	收货能力下降、拣货/发运能力下降	平均库存、容量利用率
需求端	局部需求激增、全城需求激增	lost sales、销售收入下降
系统绩效	—	利润、持有成本、恢复时间

3 本网络可能面对的中断风险

表 2: 风险清单与建议实施顺序

风险类型	发生位置	对模型的直接影响	建议
供应商产能不足	外部供应商	供应商每天只能释放正常订单量的一部分，例如 80% 或 40%	第一阶段核心
供应商完全停产	外部供应商	暂时不能释放任何订单，未完成数量进入 supplier backlog	第一阶段核心
上游运输延误	供应商-中心仓	基础提前期 {1, 2, 3} 上额外增加 1 天或 2 天	第一阶段核心
上游运输完全中断	供应商-中心仓	已生产货物暂时不能发出，进入 transport waiting queue	第一阶段核心
中心仓收发能力下降	中心仓	每天可接收或可向前置仓发运的数量下降	第二阶段可选
下游运输延误	中心仓-前置仓	原 overnight delivery 变为延误 1-2 天，需要建立前置仓在途管道	第二阶段推荐
下游运输完全中断	中心仓-前置仓	部分或全部前置仓暂时不能收到补货	第二阶段推荐
局部需求激增	前置仓服务区域	泊松均值乘以 1.25、1.5 或 2.0	组合压力测试
产品特定供应中断	供应商/SKU	仅一类或若干类产品供应量下降	后续扩展

风险类型	发生位置	对模型的直接影响	建议
共同冲击	天气、停电、信息系统	同时影响产能、运输和需求，使风险因子相关	高级扩展

第一阶段推荐范围

先只考虑“供应商可用性 A_t ”与“供应商-中心仓运输状态 U_t ”。这两个风险都发生在中心仓上游，不需要修改现有前置仓 overnight delivery 的基本逻辑；同时它们可以共同出现，能够形成有意义的复合中断情景。

4 推荐的第一阶段马尔可夫状态模型

4.1 风险作用位置

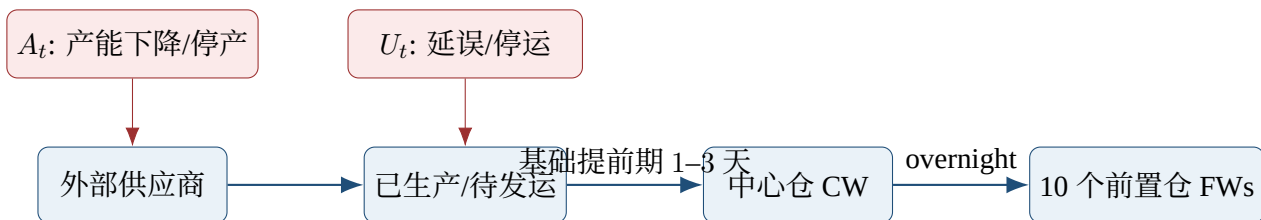


图 1: 第一阶段中断风险的作用位置

4.2 供应商可用性链 A_t

定义

$$A_t \in \{A_0, A_1, A_2, A_3\}.$$

表 3: 供应商可用性状态及运营映射

状态	含义	释放比例 $\alpha(A_t)$	解释
A_0	正常生产	1.00	所有未完成订单均可释放
A_1	轻度产能不足	0.80	每天释放约 80% 的待处理数量
A_2	严重产能不足	0.40	每天释放约 40% 的待处理数量
A_3	完全停产	0.00	当天不释放货物，订单继续积压

设 $B_{k,t}^S$ 为产品 k 在供应商处尚未生产或尚未释放的订单积压， $Q_{k,t}^C$ 为中心仓在第 t 天末新下的订单。当天待处理总量为

$$H_{k,t}^S = B_{k,t}^S + Q_{k,t}^C.$$

供应商在状态 A_t 下释放的数量为

$$R_{k,t}^S = \lfloor \alpha(A_t) H_{k,t}^S \rfloor,$$

下一天的供应商积压为

$$B_{k,t+1}^S = H_{k,t}^S - R_{k,t}^S.$$

正常状态 A_0 会清空当日所有尚未完成的订单。第一阶段假设同一供应商状态按相同比例影响所有产品；产品特定中断可以在后续研究中另行设置。

实现说明

基线模型中“供应商库存总是充足”与这里的产能中断并不冲突。可以理解为原材料或成品来源长期充足，但供应商的当日生产、质检、装车或订单释放能力暂时下降。

4.3 上游运输状态链 U_t

定义

$$U_t \in \{U_0, U_1, U_2, U_3\}.$$

表 4: 供应商至中心仓运输状态及运营映射

状态	含义	额外延误 $\delta(U_t)$	实现规则
U_0	正常运输	0 天	按基础提前期发运
U_1	轻度运输延误	1 天	新发运批次的提前期增加 1 天
U_2	严重运输延误	2 天	新发运批次的提前期增加 2 天
U_3	完全运输中断	不发运	已生产货物进入待发运队列

基础提前期继续保持原模型设定：

$$L_{k,o}^0 \sim \text{DiscreteUniform}\{1, 2, 3\}.$$

当 $U_t \in \{U_0, U_1, U_2\}$ 时，当天发运批次的实际提前期为

$$L_{k,o} = L_{k,o}^0 + \delta(U_t).$$

当 $U_t = U_3$ 时，供应商已经释放的货物不能进入运输管道，而是暂存在运输待发队列 $W_{k,t}^U$ 。运输恢复后，队列中的货物按恢复当日的运输状态发出。

第一版实现中， U_t 只影响当天新发出的批次，不追溯修改已经在途批次的剩余提前期。这样能够避免对同一批货重复增加延误。高级版本可以研究“线路中断冻结所有在途货物”，但必须另行定义，不能与本规则混用。

4.4 中心仓库存位置必须包含所有未到货数量

引入供应商积压和运输待发队列后，中心仓产品 k 的库存位置应改为

$$IP_{k,t}^C = I_{k,t}^C + B_{k,t}^S + W_{k,t}^U + \sum_{o \in \mathcal{P}_{k,t}^C} q_{k,o},$$

其中 $\mathcal{P}_{k,t}^C$ 是已经发运但尚未到达中心仓的批次集合。中心仓仍然采用原 (s, S) 规则：

$$Q_{k,t}^C = \begin{cases} S_k^C - IP_{k,t}^C, & IP_{k,t}^C \leq s_k^C, \\ 0, & IP_{k,t}^C > s_k^C. \end{cases}$$

常见误区

如果库存位置只统计“已经在途”的数量，而遗漏供应商 backlog 或运输 waiting queue，中心仓会在中断期间反复下单。中断结束后大量订单集中到达，结果可能超过中心仓容量，并人为放大牛鞭效应。

4.5 联合中断状态：16 种组合

联合状态定义为

$$Z_t = (A_t, U_t).$$

因此

$$|\mathcal{Z}| = 4 \times 4 = 16.$$

表 5: 16 个联合状态的编码示例

	U_0 正常	U_1 延误 1 天	U_2 延误 2 天	U_3 停运
A_0 正常	Z_{00}	Z_{01}	Z_{02}	Z_{03}
A_1 80%	Z_{10}	Z_{11}	Z_{12}	Z_{13}
A_2 40%	Z_{20}	Z_{21}	Z_{22}	Z_{23}
A_3 0%	Z_{30}	Z_{31}	Z_{32}	Z_{33}

例如， Z_{21} 表示供应商每天只能释放约 40% 的待处理订单，同时新发运批次额外延误 1 天； Z_{33} 表示供应商完全停产且运输也完全中断。

如果第一阶段假设两个风险链条件独立，则

$$\mathbb{P}(A_{t+1} = a', U_{t+1} = u' \mid A_t = a, U_t = u) = M_{a,a'}^A M_{u,u'}^U.$$

联合转移矩阵为

$$M^Z = M^A \otimes M^U,$$

其中 $\otimes$ 表示 Kronecker product。程序中不必显式存储 16×16 矩阵；分别从 M^A 和 M^U 抽取下一状态即可。

若使用整数编码，可定义

$$z = 4a + u, \quad a, u \in \{0, 1, 2, 3\}.$$

5 转移矩阵的设计、解释与校验**5.1 教学用示例矩阵**

供应商可用性转移矩阵可设为

$$M^A = \begin{bmatrix} 0.995 & 0.004 & 0.0008 & 0.0002 \\ 0.300 & 0.600 & 0.0800 & 0.0200 \\ 0.080 & 0.250 & 0.5500 & 0.1200 \\ 0.020 & 0.080 & 0.2500 & 0.6500 \end{bmatrix}.$$

上游运输转移矩阵可设为

$$M^U = \begin{bmatrix} 0.992 & 0.006 & 0.0015 & 0.0005 \\ 0.400 & 0.500 & 0.0800 & 0.0200 \\ 0.150 & 0.300 & 0.4500 & 0.1000 \\ 0.050 & 0.150 & 0.3000 & 0.5000 \end{bmatrix}.$$

矩阵的行表示当天状态，列表示下一天状态。每一行必须满足

$$M_{ij} \geq 0, \quad \sum_j M_{ij} = 1.$$

运输矩阵第一行的直接解释为：在今天运输正常的条件下，明天仍正常的概率为 99.2%，进入额外延误 1 天状态的概率为 0.6%，进入额外延误 2 天状态的概率为 0.15%，进入完全停运状态的概率为 0.05%。

5.2 不要混淆频率、严重程度与持续时间

表 6: 中断参数的三个不同维度

维度	由什么决定	示例
发生频率	正常状态行中的非对角概率	$M_{03}^U = 0.0005$ 是从正常直接进入停运的日概率
严重程度	状态到运营参数的映射	A_2 对应释放比例 0.40； U_2 对应额外延误 2 天
持续时间	异常状态的自转移概率	$M_{33}^U = 0.50$ 决定停运状态会持续多久

若某状态的自转移概率为 p_{jj} ，在几何持续时间假设下，该状态一次连续出现的平均长度为

$$\mathbb{E}[T_j] = \frac{1}{1 - p_{jj}}.$$

例如 $M_{33}^U = 0.50$ 时，运输完全中断的平均连续持续时间为

$$\mathbb{E}[T_{U_3}] = \frac{1}{1 - 0.50} = 2 \text{ 天}.$$

若希望平均持续 5 天，应设置

$$p_{33} = 1 - \frac{1}{5} = 0.80.$$

5.3 平稳分布与示例矩阵的长期含义

平稳分布 π 满足

$$\pi = \pi M, \quad \sum_j \pi_j = 1.$$

对上述示例矩阵，数值结果如下。

表 7: 示例矩阵的平均连续状态长度和平稳占比

链	状态	自转移概率	平均连续长度 (天)	平稳概率	长期解释
A	A_0	0.995	200.00	0.9759	供应正常
A	A_1	0.600	2.50	0.0144	轻度不足
A	A_2	0.550	2.22	0.0062	严重不足
A	A_3	0.650	2.86	0.0035	完全停产
U	U_0	0.992	125.00	0.9737	运输正常
U	U_1	0.500	2.00	0.0166	延误 1 天
U	U_2	0.450	1.82	0.0067	延误 2 天
U	U_3	0.500	2.00	0.0030	完全停运

在独立性假设下，两个链同时正常的长期概率约为

$$0.9759 \times 0.9737 \approx 0.9502,$$

即约 95.02% 的日期两个风险因子均正常，约 4.98% 的日期至少有一个因子处于异常状态。这个结果是矩阵设定的后果，不是现实数据结论。

5.4 如何构造低、中、高风险情景

可用发生率倍率 κ 调整正常状态进入异常状态的概率。对任一矩阵 M ，令

$$M_{0j}^{(\kappa)} = \kappa M_{0j}, \quad j \neq 0,$$

$$M_{00}^{(\kappa)} = 1 - \sum_{j \neq 0} M_{0j}^{(\kappa)}.$$

建议设置

$$\kappa \in \{0.5, 1, 2, 4\},$$

分别表示低风险、基准风险、中高风险和压力测试。异常状态内部的持续与恢复概率先保持不变，以便单独识别“中断发生频率”的作用。

也可以分别开展以下敏感性分析：

- (1) **频率变化**：只改变正常状态行的非对角概率；
- (2) **持续时间变化**：只改变异常状态的自转移概率；
- (3) **严重程度变化**：只改变 $\alpha(A_t)$ 或 $\delta(U_t)$ ；
- (4) **复合风险**：同时启用 A_t 和 U_t ，与单一风险结果比较。

实现说明

如果没有企业历史数据，不要把示例矩阵写成“真实发生概率”。应明确称其为 illustrative matrix 或 stress-test matrix，并通过敏感性分析证明结论不是某一组任意参数的偶然结果。

6 如何把中断过程接入现有仿真模型

6.1 每日事件顺序

初始化 $A_1 = A_0$ 、 $U_1 = U_0$ 。第 t 天的状态 A_t, U_t 在当天开始时已经确定，并影响当天末供应商对订单的处理与发运。推荐事件顺序为：

Step 1: 更新供应商到中心仓的在途批次；剩余提前期减 1，达到 0 的批次到达中心仓。

Step 2: 前一晚中心仓向前置仓发出的货物到达；保持基线模型的 overnight delivery。

Step 3: 生成前置仓泊松需求，计算销售、lost sales 和日末库存。

Step 4: 前置仓按原 (s, S) 策略下单；中心仓按既定分配规则发货。

Step 5: 中心仓计算包含全部未到货数量的库存位置，并按原 (s, S) 策略向供应商下单。

Step 6: 供应商根据 A_t 释放部分或全部未完成订单。

Step 7: 根据 U_t 决定将已释放数量发入运输管道，还是放入运输待发队列。

Step 8: 记录利润、成本、库存、缺货、积压和风险状态。

Step 9: 按 M^A 和 M^U 抽取 A_{t+1} 与 U_{t+1} 。

6.2 推荐数据结构

对每种产品 k ，至少增加三类状态变量：

$$\begin{aligned} B_{k,t}^S &: \text{供应商尚未释放的订单积压,} \\ W_{k,t}^U &: \text{已释放但因停运尚未发出的数量,} \\ \mathcal{P}_{k,t}^C &: \text{已发运的逐批在途列表.} \end{aligned}$$

在途批次建议存储为

$$(q, \ell),$$

其中 q 为批次数， ℓ 为剩余提前期。每天将 ℓ 减 1，达到 0 时到货。

6.3 完整伪代码

输入: 基线库存参数、 M^A 、 M^U 、 α 、 δ 、仿真长度 T

输出: 库存、服务、成本、利润及韧性指标

初始化中心仓和前置仓库存; $B^S \leftarrow 0$; $W^U \leftarrow 0$; 在途列表为空;

$A \leftarrow A_0$; $U \leftarrow U_0$;

for $t = 1, \dots, T$ **do**

更新所有供应商在途批次的剩余提前期, 将到期批次加入中心仓库存;

将上一晚的 CW-to-FW 发货加入各前置仓库存;

生成 $D_{i,k,t} \sim \text{Poisson}(\lambda_{i,k})$, 计算销售与 lost sales;

各前置仓执行原 (s, S) 规则并向中心仓提交请求;

中心仓按原分配规则发货, 货物在下一天早晨到达;

foreach 产品 k **do**

$IP_k^C \leftarrow I_k^C + B_k^S + W_k^U + \text{pipeline quantity}_k$;

根据中心仓 (s, S) 规则得到 Q_k^C ;

$B_k^S \leftarrow B_k^S + Q_k^C$;

$R_k^S \leftarrow \lfloor \alpha(A) B_k^S \rfloor$;

$B_k^S \leftarrow B_k^S - R_k^S$;

$W_k^U \leftarrow W_k^U + R_k^S$;

if $U \neq U_3$ **then**

抽取 $L^0 \sim \text{DiscreteUniform}\{1, 2, 3\}$;

$L \leftarrow L^0 + \delta(U)$;

将批次 (W_k^U, L) 加入供应商-中心仓在途列表;

$W_k^U \leftarrow 0$;

end

end

记录当日收入、持有成本、缺货惩罚、fill rate、库存、积压和状态;

$A \leftarrow \text{Categorical}(M_{A,\cdot}^A)$;

$U \leftarrow \text{Categorical}(M_{U,\cdot}^U)$;

end

Algorithm 1: 固定 (s, S) 策略下的中断仿真主循环

6.4 基础实现约定

(C1) 固定 (s, S) 参数不随中断状态改变。本阶段比较的是给定策略的抗冲击表现, 而不是优化策略。

(C2) 已经发运的在途批次不被后续运输状态追溯修改。

(C3) U_3 只阻止新批次进入运输管道; 运输恢复后, 待发队列统一按恢复当日状态发出。

(C4) 供应商 backlog、运输 waiting queue 和在途数量都计入中心仓库存位置。

(C5) 中心仓向前置仓未满足的请求继续采用基线模型规则, 例如不形成 backorder、次日重新计算请求。

(C6) 所有场景使用相同的需求随机种子和基础 lead-time 随机数, 尽量采用 common random numbers。

7 实验设计

7.1 实验 A: 强制中断事件

马尔可夫链中的严重中断概率较低。仅运行一年可能很少观察到停产或停运, 因此应先做可控的强制事件实验。在固定日期 τ (例如第 100 天) 强制进入某个状态并保持指定天数, 然后恢复正常。

表 8: 建议的强制中断实验

实验组	中断状态	持续时间	目的
T1	U_1 : 额外延误 1 天	1、3、5 天	识别轻度延误影响
T2	U_2 : 额外延误 2 天	1、3、5 天	识别严重延误影响
T3	U_3 : 完全停运	1、3、5、7 天	识别运输中断韧性
S1	A_1 : 释放 80%	3、7、14 天	轻度产能不足
S2	A_2 : 释放 40%	3、7、14 天	严重产能不足
S3	A_3 : 完全停产	1、3、5、7 天	供应停止影响
C1	$A_2 + U_2$	3、5 天	复合中断
C2	$A_3 + U_3$	1、3 天	极端压力测试

强制实验的优势是能够清楚比较不同严重程度和持续时间的边际影响, 并观察中断发生前、发生中和恢复后的库存轨迹。

7.2 实验 B: 长期马尔可夫运行

在长期实验中, 让 A_t 和 U_t 按转移矩阵自然演化。建议比较:

- (1) 无中断基准: 始终固定在 A_0, U_0 ;
- (2) 仅供应风险: A_t 演化、 $U_t = U_0$;
- (3) 仅运输风险: $A_t = A_0$ 、 U_t 演化;
- (4) 联合风险: A_t 与 U_t 同时演化;
- (5) 低、中、高发生率: $\kappa = 0.5, 1, 2, 4$ 。

建议使用至少 30 次独立重复; 若计算量允许, 使用 100 次。对于低概率事件, 推荐每次在 warm-up 后运行 3,650 天, 而不是只运行 365 天。结果应报告样本均值、标准差和 95% 置信区间。

7.3 核心绩效指标

网络总体 fill rate 为

$$FR = \frac{\sum_{t,i,k} Y_{i,k,t}}{\sum_{t,i,k} D_{i,k,t}}.$$

Lost-sales rate 为

$$\text{LSR} = \frac{\sum_{t,i,k} LS_{i,k,t}}{\sum_{t,i,k} D_{i,k,t}}.$$

总利润沿用基线模型定义：

$$\Pi = \sum_t \left[\sum_{i,k} p_k Y_{i,k,t} - \sum_{i,k} b_k LS_{i,k,t} - \sum_{i,k} h_k^F I_{i,k,t}^{F,\text{end}} - \sum_k h_k^C I_{k,t}^{C,\text{end}} \right].$$

建议至少输出以下指标：

- 网络总体、各产品和各前置仓的 fill rate 与 lost-sales rate；
- 总利润、销售收入、FW 持有成本、CW 持有成本和缺货惩罚；
- 平均 CW/FW 库存及容量利用率；
- CW 库存为零的天数、发生 FW 缺货的天数；
- 平均 supplier backlog、transport waiting queue 和在途数量；
- 10 个前置仓 fill rate 的最小值、标准差或极差，用于衡量服务公平性。

7.4 韧性指标

由于日需求具有随机波动，建议使用 7 日滚动 fill rate：

$$\text{FR}_t^{(7)} = \frac{\sum_{d=t-6}^t \sum_{i,k} Y_{i,k,d}}{\sum_{d=t-6}^t \sum_{i,k} D_{i,k,d}}.$$

设无中断基准的平均滚动 fill rate 为 FR^0 。中断窗口 $[\tau, \tau + H]$ 内的累计绩效损失可定义为

$$\text{LossArea} = \sum_{t=\tau}^{\tau+H} \max \left\{ 0, \text{FR}^0 - \text{FR}_t^{(7)} \right\}.$$

恢复时间可定义为

$$T_R = \min \left\{ h : \text{FR}_{\tau+h}^{(7)} \geq 0.95\text{FR}^0 \text{ 且连续 3 天满足} \right\}.$$

如果到仿真结束仍未恢复，应报告为右删失结果，而不是任意赋予一个恢复时间。

8 可选扩展与状态数量增长

8.1 下游运输中断

中心仓到前置仓的基准补货为 overnight delivery。可定义

$$D_t \in \{D_0, D_1, D_2, D_3\},$$

例如：

- D_0 ：所有前置仓正常次日到货；

- D_1 : 固定抽取 2 个受影响前置仓, 延误 1 天;
 - D_2 : 固定抽取 5 个受影响前置仓, 延误 2 天;
 - D_3 : 中心仓当晚不能向任何前置仓发货。
- 加入该风险后, 前置仓必须跟踪在途和已接受未发出的订单, 并按库存位置而不是单纯在手库存执行 (s, S) :

$$IP_{i,k,t}^F = I_{i,k,t}^F + P_{i,k,t}^F + O_{i,k,t}^F.$$

同一次局部中断期间, 受影响前置仓集合应保持不变, 不能每天重新抽取。

8.2 需求激增

需求激增不是供应中断, 但可作为独立压力因子:

$$G_t \in \{G_0, G_1, G_2\}, \quad \chi(G_t) \in \{1, 1.25, 1.50\}.$$

需求变为

$$D_{i,k,t} \sim \text{Poisson}(\chi_{i,k,t} \lambda_{i,k}).$$

可以设置全城激增、局部 2–3 个前置仓激增, 或者仅 P1、P2 高频产品激增。

8.3 状态爆炸问题

表 9: 逐步增加风险因子后的联合状态数量

风险因子	状态数量	建议
仅上游运输 U	4	最简入门
供应商 A + 上游运输 U	$4 \times 4 = 16$	第一阶段推荐
再加下游运输 D	$4 \times 4 \times 4 = 64$	第二阶段
再加需求冲击 G	$4 \times 4 \times 4 \times 3 = 192$	不建议一开始使用

风险因子越多, 并不代表模型越好。应优先保证每个状态有明确的运营含义、转移概率可解释、代码逻辑可验证, 并且实验能够识别各因素的影响。

9 学生交付清单与建议工作流程

9.1 交付清单

最终提交内容

1. 风险识别表: 列出候选风险、作用位置、模型参数和是否纳入第一阶段。

2. 状态定义表: 明确 A_0 – A_3 、 U_0 – U_3 的含义及参数映射。

3. 转移矩阵: 给出 M^A 、 M^U , 并解释每一行的逻辑。

4. 矩阵验证：行和检查、平均持续时间、平稳分布和至少一条 1,000 天状态路径图。
5. 仿真代码：新增 supplier backlog、transport waiting queue、逐批 pipeline 和状态抽样模块。
6. 单元测试：验证无中断时结果与原基线模型一致；验证 U_3 时不发运；验证 A_3 时不释放；验证库存位置没有遗漏未到货数量。
7. 实验结果：强制事件实验、长期马尔可夫实验、均值与 95% 置信区间。
8. 讨论：比较频率、严重程度和持续时间的影响，并解释最脆弱的产品、节点和绩效指标。

9.2 建议的三阶段工作流程

阶段 1: 状态与矩阵。先独立生成状态路径，确认转移逻辑、状态频率和持续时间正确，不接入库存模型。

阶段 2: 模型接入与单元测试。先只启用上游运输链 U_t ，再启用供应商链 A_t ，最后形成联合状态。

阶段 3: 实验与解释。先做强制事件，再做长期随机实验；先报告运营轨迹，再报告汇总统计。

10 常见错误检查表

编码前必须检查

1. 把“CW 缺货”写成外生马尔可夫状态，而不是仿真结果。
2. 转移矩阵某行不等于 1，或由于四舍五入出现负概率。
3. 混淆“进入完全中断的概率”和“完全中断持续的天数”。
4. 每一天都给同一在途批次重复增加 1 天或 2 天延误。
5. 运输停运时直接删除货物，而不是进入待发队列。
6. 供应产能不足时直接取消未满足部分，却没有说明 lost order 还是 backlog。
7. 中心仓库存位置遗漏 supplier backlog 或 transport waiting queue，导致重复下单。
8. 只运行 365 天就评价极低概率中断，导致结果主要由是否“碰巧发生事件”决定。
9. 不同情景使用不同需求随机数，导致中断效应与随机需求差异混在一起。
10. 加入下游延误后仍按 FW 在手库存下单，没有把在途和已接受订单计入库存位置。

A 用于矩阵校验的 Python 示例

Listing 1: Transition-matrix validation and joint-state construction

```
import numpy as np

MA = np.array([
    [0.995, 0.004, 0.0008, 0.0002],
    [0.300, 0.600, 0.0800, 0.0200],
    [0.080, 0.250, 0.5500, 0.1200],
    [0.020, 0.080, 0.2500, 0.6500],
], dtype=float)

MU = np.array([
    [0.992, 0.006, 0.0015, 0.0005],
    [0.400, 0.500, 0.0800, 0.0200],
    [0.150, 0.300, 0.4500, 0.1000],
    [0.050, 0.150, 0.3000, 0.5000],
], dtype=float)

def validate_transition_matrix(M: np.ndarray) -> None:
    if M.ndim != 2 or M.shape[0] != M.shape[1]:
        raise ValueError("M must be square")
    if np.any(M < -1e-12):
        raise ValueError("M contains a negative probability")
    if not np.allclose(M.sum(axis=1), 1.0, atol=1e-12):
        raise ValueError("Each row of M must sum to one")

def stationary_distribution(M: np.ndarray) -> np.ndarray:
    # Solve  $\pi M = \pi$  and  $\sum(\pi) = 1$ .
    n = M.shape[0]
    A = np.vstack([M.T - np.eye(n), np.ones(n)])
    b = np.append(np.zeros(n), 1.0)
    pi, *_ = np.linalg.lstsq(A, b, rcond=None)
    return pi

validate_transition_matrix(MA)
validate_transition_matrix(MU)

pi_A = stationary_distribution(MA)
pi_U = stationary_distribution(MU)
mean_spell_A = 1.0 / (1.0 - np.diag(MA))
mean_spell_U = 1.0 / (1.0 - np.diag(MU))

# State index:  $z = 4 * a + u$ .
M_joint = np.kron(MA, MU)
```



```

validate_transition_matrix(M_joint)

print("pi_A:", pi_A)
print("pi_U:", pi_U)
print("mean spells A:", mean_spell_A)
print("mean spells U:", mean_spell_U)
print("joint normal probability:", pi_A[0] * pi_U[0])

```

B 用于状态抽样与管道更新的 Python 骨架

Listing 2: Minimal state sampling and per-batch pipeline logic

```

from dataclasses import dataclass
import numpy as np

@dataclass
class Batch:
    quantity: int
    remaining_lead_time: int

rng = np.random.default_rng(2026)
alpha = np.array([1.0, 0.8, 0.4, 0.0])
extra_delay = np.array([0, 1, 2, -1]) # -1 means no dispatch

A_state = 0
U_state = 0
supplier_backlog = np.zeros(5, dtype=int)
transport_waiting = np.zeros(5, dtype=int)
pipeline = [[] for _ in range(5)]

def receive_pipeline(product: int) -> int:
    arrivals = 0
    surviving_batches = []
    for batch in pipeline[product]:
        batch.remaining_lead_time -= 1
        if batch.remaining_lead_time <= 0:
            arrivals += batch.quantity
        else:
            surviving_batches.append(batch)
    pipeline[product] = surviving_batches
    return arrivals

def process_supplier_order(product: int, new_order: int) -> None:
    global supplier_backlog, transport_waiting

```

```
supplier_backlog[product] += int(new_order)
released = int(np.floor(alpha[A_state] * supplier_backlog[product]))
supplier_backlog[product] -= released
transport_waiting[product] += released

if U_state != 3 and transport_waiting[product] > 0:
    base_lead_time = int(rng.integers(1, 4)) # 1, 2, or 3
    lead_time = base_lead_time + int(extra_delay[U_state])
    pipeline[product].append(
        Batch(int(transport_waiting[product]), lead_time)
    )
    transport_waiting[product] = 0

def sample_next_states() -> None:
    global A_state, U_state
    A_state = int(rng.choice(4, p=MA[A_state]))
    U_state = int(rng.choice(4, p=MU[U_state]))
```

最后说明

学生可以采用与本文不同的状态数量或转移矩阵，但必须回答四个问题：每个状态改变了什么运营参数？每个转移概率如何解释？平均持续时间和平稳占比是否合理？该设定能否在代码中无歧义地执行？只要这四点清楚，模型就具备进一步研究和扩展的基础。